數論演算法

從整除、不變量與同餘出發，建立最大公因數、模方程、快速冪、質數測試與 RSA。

整除與同餘

$a \equiv b \pmod n$ 意味 $n \mid (a-b)$，$n>0$。例 $29 \equiv 5 \pmod{12}$，因 $29-5=2 \cdot 12$。加法與乘法可逐項取餘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歐幾里得算法

例 gcd(252,198)：252=1・198+54；198=3・54+36；54=1・36+18；36=2・18+0，所以 gcd=18。

$$252 = 1 \times 198 + 54$$

$$198 = 3 \times 54 + 36$$

$$54 = 1 \times 36 + 18$$

$$36 = 2 \times 18 + 0$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大公因數不變量

a=qb+r 時，a、b 的公因數恰為 b、r 的公因數，因 r=a−qb 且 a=qb+r。非雙零輸入先取絕對值，最後非零餘數就是 gcd。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

擴展歐幾里得表

保持 r=252s+198t。各列 (r,s,t)：(252,1,0)，(198,0,1)，(54,1,−1)，(36,−3,4)，(18,4,−5)，(0,−11,14)。因此 18=4・252−5・198。

$$252 = 1\times252 + 0\times198$$

$$198 = 0\times252 + 1\times198$$

$$54 = 1\times252 - 1\times198$$

$$36 = -3\times252 + 4\times198$$

$$18 = 4\times252 - 5\times198$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

係數更新與終止

若前兩列為 (r_0, s_0, t_0)、(r_1, s_1, t_1)，商 $q = \lfloor r_0 / r_1 \rfloor$，下一列是 $(r_0 - q r_1, s_0 - q s_1, t_0 - q t_1)$。僅在 $r_1 > 0$ 時更新，此時 $0 \le r_0 - q r_1 < r_1$。每列維持線性組合，正餘數遞減而終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數輸入與 Bézout 係數

若 (a,b)=(0,0)，定義 gcd=0，取 x=y=0。其餘情況從 $r_0=|a|$、$r_1=|b|$ 開始，對 |a|、|b| 的係數列為 (1,0)、(0,1)。$r_1=0$ 時先停止，不做除法；最後非零 r_0 是 gcd(a,b)。

若 $r_0=s|a|+t|b|$，回填 x=s・sign(a)、y=t・sign(b)，其中 sign(0)=0。則 ax+by=gcd(a,b)，包含任一輸入為零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

帶負號的例子

(7,−3) 取 (7,3)：7=2・3+1；3=3・1+0。回代 1=7−2・3=1・7+2・(−3)，故 (x,y)=(1,2)。

(−12,18) 取 (12,18)：12=0・18+12；18=1・12+6；12=2・6+0。回代 6=18−12=1・(−12)+1・18，故 (x,y)=(1,1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

零輸入的例子

(0,−5) 取 (0,5)：0=0・5+0，接著停止。gcd=5=0・0+(−1)・(−5)，故 (x,y)=(0,−1)。

(0,0) 不做除法。依約定 gcd(0,0)=0=0・0+0・0，取 (x,y)=(0,0)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid 的最壞情況

令 a≥b>0，Euclid 使用帶餘除法。若有 k 次非零餘數，逆推最小可能輸入得到相鄰 Fibonacci 數；因此 $b \geq F_{k+1}$，而 $F_j \geq \varphi^{j-2}$（j≥2，φ=(1+√5)/2）。所以除法次數是 O(log(b+1))；相鄰 Fibonacci 數在輸入增大時達到 Θ(log b)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼 Fibonacci 出現？

設 g=gcd(a,b)>0，先把所有餘數除以 g，則最後兩個標準化餘數為 g/g=1 和 0。從標準化序列的終點 0,1 向前，每一步 $r_{i-1} = q_i r_i + r_{i+1}$，商 $q_i \geq 1$，所以 $r_{i-1} \geq r_i + r_{i+1}$。用商 1 反推得到 Fibonacci 下界。原始餘數最後是 g,0，故不假設 g=1。例 34=1・21+13、21=1・13+8、13=1・8+5、8=1・5+3、5=1・3+2、3=1・2+1、2=2・1+0。

反推最小餘數

每個值至少是右側兩值之和



商為 1 時取等號

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

除法次數與 bit 成本

令兩輸入最多各有 B 位。若採用傳統長除法，一次 B 位除法 $O(B^2)$ bit operations，加上 $O(B)$ 次除法，得到簡單上界 $O(B^3)$。這是保守上界，不等於最佳實作的複雜度；以除法運算為單位的 $O(B)$ 不能直接當作 bit 成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid 兩步減半與逆元證書

對 a≥b>0，第一個餘數 r=a mod b。若 r≤b/2，次參數一步已至少減半；若 r>b/2，因 b/2<r<b，下一次商為 1，下一餘數 b mod r=b−r<b/2。因此每至多兩次除法，新的次參數不超過原來的一半，或演算法先終止。這給出 O(log(b+1)) 次除法，不把大整數除法的 bit 成本當成常數。

例 391=299+92、299=3・92+23、92=4・23+0，所以 gcd=23。回代 23=299−3(391−299)=4・299−3・391。因 gcd(299,391)=23≠1，299 沒有模 391 的乘法逆元；若存在 299u≡1 (mod 391)，則 299u−391v=1 的左側卻被 23 整除，矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性同餘的可解性

$ax \equiv b \pmod n$ 等價於存在整數 y 使 $ax+ny=b$。令 $d=\gcd(a,n)$。$n>0$ 時，可解的必要且充分條件是 $d \mid b$，且可解時模 n 恰有 d 個不同解：必要性由公因數整除左側；充分性由 Bézout 等式乘以 b/d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

列出全部同餘解

解 $14x \equiv 30 \pmod{100}$。$d=2$，化為 $7x \equiv 15 \pmod{50}$，$7^{-1} \equiv 43$，故 $x \equiv 45 \pmod{50}$。模 100 恰有兩解 45、95；代入皆餘 30。

$$14x \equiv 30 \pmod{100}$$

$$7x \equiv 15 \pmod{50}$$

$$x \equiv 45 \pmod{50}$$

$$x = 45,\ 95 \pmod{100}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

互逆元素

a 在模 n 下有逆元當且僅當 gcd(a,n)=1。例 7・43=301≡1 (mod 50)。擴展歐幾里得找出的 Bézout 係數即逆元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

模線性方程：壓縮表示與輸出成本

求 $ax \equiv b \pmod n$，$n \geq 1$。令 $g=\gcd(a,n)$。若 $g \nmid b$ 無解；若 $g \mid b$，將方程與模數除以 g。若 $step=n/g>1$，此時 a/g 與 step 互質，可用逆元求一個 x_0；若 $step=1$，直接取 $x_0=0$，不需模1逆元。所有模 n 解恰是 $x_0+j \cdot step$（$0 \leq j<g$），其中 $step=n/g$，x_0 取 $0 \leq x_0<step$。用三元組 $(x_0,step,g)$ 可表示全部解。

例 $24x \equiv 36 \pmod{60}$：$g=12$，約分得 $2x \equiv 3 \pmod 5$，$x_0=4$、$step=5$，壓縮結果 $(4,5,12)$，代表 4、9、…、59 共12個解。$0x \equiv 0 \pmod n$ 對所有 n 個剩餘類成立，其三元組為 $(0,1,n)$，包含 $n=1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

壓縮與逐一列出的成本

若 a、b 先模 n 化，三元組中每個數至多 n，所需空間是 $O(\log(n+1))$ bits。這只是輸出表示大小，不包含求 gcd 或逆元的運算成本。

若要求逐一列出 g 個解，至少需 $\Omega(g)$ 次輸出；對 $0x \equiv 0 \pmod{n}$，$g=n$，若用一般二進位逐一寫出所有代表，總輸出長度為 $\Theta(n \log(n+1))$ bits。當 n 以 $\beta=\lceil \log_2(n+1) \rceil$ bits 給出時，n 可對 β 指數增長；因此壓縮輸出和逐一列舉不是相同成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

模算術的除法代表乘逆元

模 n 只有與 n 互質的元素才有乘法逆元。若 $b \in U(n)$，記 a/b 為 $a \cdot b^{-1} \pmod n$，它不是先做整數除法再取餘。例模 15 下，$7 \cdot 13 = 91 \equiv 1$，故 $4/7 \equiv 4 \cdot 13 \equiv 7 \pmod{15}$。如果分母不是 unit，就不能直接取消；應解線性同餘並檢查 gcd 的可解條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中國剩餘定理

若模數兩兩互質，給定各餘數有唯一解模各模數乘積。例 $x \equiv 2 \pmod 3$、$x \equiv 3 \pmod 5$：3 的倍數加 2 依序 2、5、8，故 $x \equiv 8 \pmod{15}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非互質模數的相容性

$x \equiv a \pmod m$、$x \equiv b \pmod n$ 可解當且僅當 $\gcd(m,n) \mid (b-a)$，解唯一模 $\operatorname{lcm}(m,n)$。令 $x=a+mt$，轉成 $mt \equiv b-a \pmod n$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非互質例子

$x \equiv 2 \pmod 6$、$x \equiv 5 \pmod 9$：gcd=3 整除差 3，$6t \equiv 3 \pmod 9$，得 $t \equiv 2 \pmod 3$，故 $x \equiv 14 \pmod{18}$。若第二餘數為 4，差 2 不可被 3 整除，無解。

$$x = 2 + 6t$$

$$6t \equiv 3 \pmod 9$$

$$t \equiv 2 \pmod 3$$

$$x \equiv 14 \pmod{18}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中國剩餘定理：互質模數

給定兩兩互質 $m_1,\cdots,m_k>1$ 與任意 a_i，$M=\prod m_i$，$M_i=M/m_i$。因 $\gcd(M_i,m_i)=1$，取 $y_i=M_i^{-1} \bmod m_i$；解為 $x\equiv\Sigma a_iM_iy_i \pmod M$。

對第 i 個模數，只有第 i 項是 a_i，其餘項均為 0，因此構造確實滿足全部條件。若 x,x' 都是解，每個 m_i 均整除 $x-x'$；兩兩互質故 M 整除差，故解模 M 唯一。

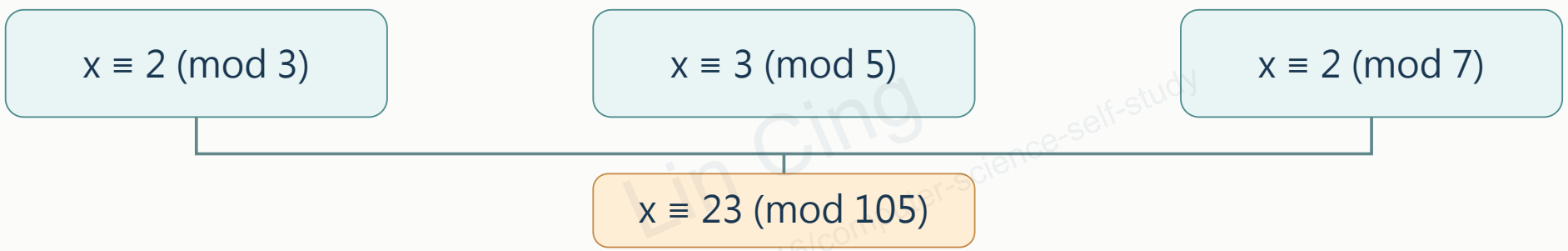
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT 例子

$x \equiv 2 \pmod 3$、$x \equiv 3 \pmod 5$、$x \equiv 2 \pmod 7$。$M=105$，$(M_1,M_2,M_3)=(35,21,15)$，其逆元分別為 $(2,1,1)$。

$x \equiv 2 \cdot 35 \cdot 2+3 \cdot 21+2 \cdot 15=233 \equiv 23 \pmod{105}$。檢查 23 的餘數分別是 2、3、2。

同一個整數滿足三條限制



23 除以 3、5、7 的餘數依序是 2、3、2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不互質的 CRT

兩條同餘 $x \equiv a \pmod m$、$x \equiv b \pmod n$ 有解，若且唯若 $d=\gcd(m,n)$ 整除 $b-a$。必要性來自 $x-a$ 為 m 的倍數且 $x-b$ 為 n 的倍數。充分性：令 $x=a+mt$，解 $(m/d)t \equiv (b-a)/d \pmod{n/d}$；因 m/d 與 n/d 互質，t 唯一模 n/d，故 x 唯一模 $\operatorname{lcm}(m,n)$。多條限制可依序合併，遇不相容即無解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

模冪演算法

m>1、e≥0。以 accumulator=1 開始，從最高位到最低位掃描 e 的全部位元：先 acc=acc^2 mod m，位元為 1 再 acc=acc・a mod m。e=0 時結果 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

模冪逐位追蹤

算 $7^{13} \bmod 23$，$13=(1101)_2$。acc 從 1 開始，第一位 1 先平方得 1，再乘 7 得 7；第二位 1 算 $7^2 \bmod 23=3$，再乘 7 得 21。各位後依序為 7、21、4、20。故結果 20。

bits: 1 1 0 1

acc: 1 → 7 → 21 → 4 → 20

$7^{13} \bmod 23 = 20$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

模冪不變量

已處理前綴代表整數 k 時，acc≡a^k (mod m)。新位 b 把前綴變成 2k+b；平方後乘 a^b 恰得新指數。掃描 O(log(e+1)) 位，這是模乘次數而非位元運算成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

右到左模冪與不變量

輸入 $A \in \mathbb{Z}$、$B \geq 0$、$n \geq 1$。初始化 $r=1 \bmod n$、$a=A \bmod n$、$b=B$；當 $b>0$，若 b 為奇數先令 $r \leftarrow ra \bmod n$，再令 $a \leftarrow a^2 \bmod n$、$b \leftarrow \lfloor b/2 \rfloor$。不變量是 $r \cdot a^b \equiv A^B \pmod n$。

若 $b=2q$，更新後 $r(a^2)^q = r \cdot a^b$；若 $b=2q+1$，更新後 $(ra)(a^2)^q = r \cdot a^b$。每輪 b 減半，終止時 $b=0$，因此 $r \equiv A^B$。b 初始為 0 時直接回傳 $1 \bmod n$；特別地 $n=1$ 時回傳 0。非互質底數同樣適用，因為只做乘法、不需逆元。負指數不是此演算法的輸入；若要定義，需另要求底數可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

右到左模冪：逐輪核對

計算 $7^{19} \bmod 23$，初態 (r,a,b)=(1,7,19)。每列由上一列依不變量更新；最後得到 11。

處理前 (r,a,b)	處理後 (r,a,b)
(1,7,19)	(7,3,9)
(7,3,9)	(21,9,4)
(21,9,4)	(21,12,2)
(21,12,2)	(21,6,1)
(21,6,1)	(11,13,0)

例如第三列 b=4 為偶數，r 保持 21，a 平方為 $9^2 \equiv 12$，b 除以 2 得 2。

有限閉合子集與元素階

有限群 G 的非空子集 S 若對群運算封閉，就是子群。任取 a∈S；序列 a,a^2,⋯都在 S 中，有限性保證 $a^i=a^j$（1≤i<j），在 G 中消去 a^i 得 $a^{j-i}=e\in S$。若 j−i>1，$a^{j-i-1}\in S$ 是 a 的逆；若 j−i=1 則 a=e，其逆也在 S。這裡「有限」不能刪除。

元素 a 的 order 是使 $a^t=e$ 的最小正整數 t；e,a,⋯,a^{t-1} 互異，所有冪按指數模 t 重複。因此 $|\langle a\rangle|=t$，由 Lagrange 得 t | |G|，進而 $a^{|G|}=e$。例在加法群 Z_8 中，2 生成 {0,2,4,6}，order 是 4；在乘法群 U(7) 中，2 生成 {1,2,4}，order 是 3。兩者運算不同，符號 ⟨2⟩ 的意義也不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lagrange 定理：陪集如何分組？

設 G 是有限群、H 是子群。對 $a \in G$，左陪集 $aH=\{ah : h \in H\}$。映射 $h \mapsto ah$ 是雙射：左乘 a^{-1} 可還原 h，所以每個左陪集都有 $|H|$ 個元素。

不同左陪集不是完全相同就是不相交：若 $ah_1=bh_2$，則 $b^{-1}a=h_2h_1^{-1} \in H$，因而 $aH=bH$。每個元素 a 都屬於 aH（因單位元在 H），所以左陪集把 G 分割成大小相同的區塊。故 $|G|=[G:H]|H|$，特別是 $|H|$ 整除 $|G|$。對 $H=\langle g \rangle$，便得 $\operatorname{ord}(g) \mid |G|$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler φ 的質數冪公式與積性

對質數 p、整數 $e \geq 1$，$1, \cdots, p^e$ 中恰有 p^{e-1} 個 p 的倍數，其餘均與 p^e 互質，因此 $\phi(p^e) = p^e - p^{e-1} = p^{e-1}(p-1)$。

若 $\gcd(m,n)=1$，CRT 給出剩餘類的雙射；x 與 mn 互質，當且僅當它模 m、模 n 的分量分別是 unit。故 $U(mn) \cong U(m) \times U(n)$，取數量得 $\phi(mn)=\phi(m)\phi(n)$。對一般 n 可先分解成互質的質數冪再相乘。例 $72 = 8 \cdot 9$，$\phi(72)=\phi(8)\phi(9)=4 \cdot 6=24$；約定 $\phi(1)=1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有限群、Euler 定理與離散對數

模 n 的可逆元素 U(n) 在乘法下成群，大小為 φ(n)。對 a∈U(n)，乘以 a 是 U(n) 的置換；相乘並消去所有可逆因子，得到 $a^{\varphi(n)} \equiv 1 \pmod{n}$。更精確地，a 的階 $\mathrm{ord}_n(a)$ 整除 φ(n)，因其生成的子群大小整除群大小。

若 g 的階恰為 φ(n)，g 是模 n 的 primitive root，且 U(n)=⟨g⟩。給定 h∈⟨g⟩，離散對數是滿足 $g^x=h$ 的 x，唯一模 ord(g)；若 h 不在子群內則無解。不是每個模數都有 primitive root。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

群與離散對數例子

模 7 的 $U(7)=\{1,2,3,4,5,6\}$。3 的冪依次為 3、2、6、4、5、1，因此 $\mathrm{ord}_7(3)=6$，3 為 primitive root；$\log_3 5 \equiv 5 \pmod 6$。相反，2 的冪為 2、4、1，生成子群 $\{1,2,4\}$；$2^x \equiv 3 \pmod 7$ 無解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何模質數一定有生成元？

令 $G=F_p$ 的非零乘法群。對 G 中所有元素的 order 取最小公倍數 L。對 L 每個質因數 ℓ 的最高冪 ℓ^a，挑一個 order 含此冪的元素，再取適當次方得到 order 恰為 ℓ^a 的元素。

不同質數冪的 order 互質。因 G 交換，這些元素相乘，其 order 為它們 order 的乘積 L。證明：若乘積的 k 次方為 1，每個因子的 k 次方同時位於自己所生成的子群及其餘因子所生成的子群。兩群的階互質，交集只有單位元，故每個 order 都整除 k；反向 L 次方顯然為 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用多項式根數完成證明

所有 $x \in G$ 均滿足 $x^L = 1$。體上非零 L 次多項式 $X^L - 1$ 最多有 L 個根，因此 $p - 1 \le L$。另一方面，由 Lagrange 定理每個元素的 order 整除 $p - 1$，所以 L 也整除 $p - 1$，特別是 $L \le p - 1$。故 $L = p - 1$，上一頁構造的元素生成 G。

根數上界依賴係數所在結構是「體」；不能把這項證明直接套用到任意模合數的剩餘環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

將生成元從 p 提升到 p^2

p 為奇質數，g 是模 p 的 primitive root。比較 g 與 g+p；二項式定理給 $(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p \pmod{p^2}$。第二項恰為非零的 p 倍數模 p^2，因此兩者至少一個 h 滿足 $h^{p-1} \not\equiv 1 \pmod{p^2}$。又 $h^{p-1} \equiv 1 \pmod{p}$，可寫成 1+cp，其中 $p \nmid c$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

提升至所有奇質數冪

若 p 為奇質數且 $v_p(z-1)=j\geq 1$，二項式展開 $z^p-1=(1+(z-1))^p-1$：首項的 p-adic valuation 是 j+1，所有其餘項 valuation 更高，因此 $v_p(z^p-1)=j+1$。歸納得到 $v_p(h^{(p-1)p^k}-1)=k+1$。

h 在模 p^e 的 order 是 (p−1) 的倍數，且整除 $\phi(p^e)=(p-1)p^{e-1}$。上式排除少於 e−1 個 p 因子，所以 order 恰為 $(p-1)p^{e-1}$，故 $U(p^e)$ 是 cyclic。例 p=3，g=2；$2^2=4\not\equiv 1 \pmod 9$，所以 2 生成 $U(3^e)$ 的所有元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些模數有 primitive root？

對 $n>1$，$U(n)$ 是 cyclic 當且僅當 $n=2$、4、p^e 或 $2p^e$，其中 p 為奇質數、$e\geq 1$。$U(2)$、$U(4)$ 可直接列舉；$U(p^e)$ 已證 cyclic，而 CRT 給 $U(2p^e)\cong U(2)\times U(p^e)$，前一分量只有一個元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排除其餘模數

若 $8|n$，寫 $n=2^a m$，其中 $a\geq 3$ 且 m 為奇數。任取 $b\in U(8)$，選 b 的奇整數代表，先視為模 2^a 的 unit。當 $m>1$，用互質的模數 2^a 與 m 以 CRT 解 $x\equiv b \pmod{2^a}$、$x\equiv 1 \pmod{m}$；當 $m=1$，直接取 $x=b$。所得 x 與 n 互質且 $x\equiv b \pmod 8$，故 $U(n)\to U(8)$ 的自然投影滿射。$U(8)=\{1,3,5,7\}$ 的每個元素平方皆為 1，卻有四個元素，因此非 cyclic。cyclic 群的同態像仍 cyclic，故 $U(n)$ 不可能 cyclic。

若 n 有兩個相異奇質因數，CRT 可各自選 ± 1，得到至少四個平方為 1 的 unit；若 $4|n$ 且 n 另有奇質因數，$U(4)$ 與奇質數冪分量亦產生至少四根。有限 cyclic 群的 $x^2=1$ 至多有兩個解，故這些情況均被排除，完整涵蓋分類以外的 n。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

模 121 的生成元：不能只看模 11

模 11 時，2 的 order 為 10。但要提升到 $11^2=121$，還需檢查 $2^{10}=1024\equiv 56\not\equiv 1 \pmod{121}$。模 121 的 order 是模 11 的 order 10 的倍數，且整除 $\phi(121)=110$；因此只可能是 10 或 110。10 已被排除，故 order=110，2 確實生成 $U(121)$。

這次檢查在模 p^2 不等於 1，才可搭配前述 valuation 推導提升至更高奇質數冪；不能把「模 p 有生成元」直接當成任意選取的整數代表在模 p^2 仍是生成元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

奇質數冪的平方根只有 ±1

設 p 為奇質數、$e \geq 1$，若 $x^2 \equiv 1 \pmod{p^e}$，則 $p^e \mid (x-1)(x+1)$。由 $\gcd(x-1, x+1)$ 整除 2 可知 p 不可能同時整除兩個因子。因此 p^e 的全部 p 因子都落在同一側，得到 $x \equiv 1$ 或 $-1 \pmod{p^e}$。反向這兩個值確實都是根；因 p 為奇數，兩者不同。

例模 25 僅有 1、24 兩個平方根。模合數可能有更多：模 15 的 1、4、11、14 都平方等於 1，這正是 CRT 各分量獨立取符號的結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非平凡平方根如何給出因數？

給定奇數 $n>1$，若 $x^2\equiv 1 \pmod n$ 但 $x\not\equiv 1、-1 \pmod n$，令 $d_-=\gcd(x-1,n)$、$d_+=\gcd(x+1,n)$。兩者都小於 n，否則 $x\equiv \pm 1$；若 $d_-=1$，從 $n\mid(x-1)(x+1)$ 可消去 $x-1$，推得 $n\mid(x+1)$，矛盾。$d_+=1$ 亦同理。故兩者皆為非平凡因數。

例 $n=77$、$x=34$：$34^2=1156\equiv 1 \pmod{77}$，而 $\gcd(33,77)=11$、$\gcd(35,77)=7$。以 CRT 看，$34\equiv 1 \pmod{11}$、$34\equiv -1 \pmod 7$；不同分量選了不同符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

質數與合數

試除至 $\sqrt{n}$ 可判定整數 $n \geq 2$，但對長輸入所需步數可能很大。例 $91 = 7 \cdot 13$ 是合數；97 只需試除不超過 $\sqrt{97}$ 的質數 2、3、5、7，皆不整除，故為質數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller–Rabin 前置條件

$n<2$ 判定不是質數；$n=2$ 或 3 判質數；偶數 $n>2$ 判合數。對奇數 $n\geq 5$ 寫 $n-1=2^s d$，d 為奇數，選底數 $a\in\{2,\cdots,n-2\}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

見證測試

算 $x=a^d \bmod n$。若 $x=1$ 或 $n-1$，此輪通過；否則重複平方至多 $s-1$ 次，若遇 $n-1$ 則通過；否則 a 是合數見證。例 $n=15$，$a=2$，$d=7$、$s=1$，$2^7 \bmod 15=8$，未通過。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

質數不會被誤判

奇質數 p 中，$x^2 \equiv 1 \pmod p$ 只有 ± 1，因 $p \mid (x-1)(x+1)$。結合費馬定理 $a^{p-1} \equiv 1$，可知合法底數必通過測試。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機錯誤率與限制

對固定奇合數 $n \geq 5$，每輪從 $2 \cdots n-2$ 獨立均勻抽底數，通過 t 輪仍為合數的機率至多 4^{-t}。這是對固定合數與隨機選擇的條件機率，不是任意輸入為合數的後驗機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fermat 偽質數：341

341=11・31 是合數，但 $2^{10}=1024\equiv 1 \pmod{341}$，故 $2^{340}=(2^{10})^{34}\equiv 1$。對底數 2 的 Fermat 測試會接受 341，卻不能因此認定它是質數。

Miller–Rabin 取 $340=2^2\cdot 85$；$2^{85}=(2^{10})^8\cdot 2^5\equiv 32$，下一次平方 $32^2=1024\equiv 1 \pmod{341}$。初值 32 既非 1 亦非 340，平方前也未見 −1，所以測試拒絕 341。這裡 32 是非平凡平方根；$\gcd(32-1,341)=31$，$\gcd(32+1,341)=11$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Carmichael 數：561

561=3・11・17 是合數，且 560 同時是 3−1=2、11−1=10、17−1=16 的倍數。若 gcd(a,561)=1，由 Fermat 定理在三個質數模數下分別有 $a^{560}\equiv 1$；CRT 因而給 $a^{560}\equiv 1 \pmod{561}$。因此每一個與 561 互質的底數都通過 Fermat 測試；若底數不互質，這項結論不適用。

對 Miller–Rabin，$560=2^4\cdot 35$，底數 2 的序列為 $2^{35}\equiv 263$，接著平方依次得 166、67、1（mod 561）。初值不是 1，任何可檢查的項也不是 560，因此底數 2 揭露 561 是合數。注意 Fermat 全部互質底數均會受騙，不代表 strong test 也全部受騙。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strong pseudoprime：2047

2047=23・89 是合數，2046=2・1023，s=1、d=1023。由 $2^{11}=2048\equiv 1 \pmod{2047}$ 且 1023=11・93，得到 $2^{1023}\equiv 1$。因此 Miller–Rabin 以底數 2 的這一輪會接受 2047；2 是此合數的 strong liar。

三例對照的是固定底數的一輪結果。Miller–Rabin 的隨機誤判界來自依規定獨立、均勻抽取多個底數；固定使用底數 2 並不具備該機率保證。

同一底數 2，三個合數的結果

數字	Fermat	Miller–Rabin	依據
341 = 11·31	接受	拒絕	32 → 1
561 = 3·11·17	接受	拒絕	263 → 166 → 67 → 1
2047 = 23·89	接受	接受	$2^{1023} \equiv 1$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

65 的 strong-liar 精確清單

$65=5\cdot 13$，$64=2^6\cdot 1$，所以 $s=6$、$d=1$；$\phi(5)=2^2\cdot 1$、$\phi(13)=2^2\cdot 3$。兩分量各有 $\gcd(1,u_i)=1$、$t_i=2$，故 $L=1+1+4=6$。

初值為 1 只給 $a=1$；初值為 -1 給 $a=64$。下一層需 $a^2\equiv -1 \pmod{65}$：模 5 的根是 2、3，模 13 的根是 5、8，四組 CRT 解為 8、18、47、57。完整 liar 集合為 $\{1,8,18,47,57,64\}$，其中實際底數區間 $2,\dots,63$ 只留下四個，通過比例為 $4/62=2/31$。底數 2 的六個檢查值為 2、4、16、61、16、61，既未在初值得到 1，也未出現 64，故它是 witness。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller–Rabin 的 1/4 界：計數準備

令奇合數 $n = \prod_i p_i^{e_i}$，n−1=2ˢd（d 奇）。對每個 i，$\varphi(p_i^{e_i}) = 2^{t_i} u_i$（uᵢ 奇），令 A=∏gcd(d,uᵢ)、t=min(s,t₁,⋯,tₖ)。奇質數冪的乘法群是循環群；在大小為 2ᵗⁱuᵢ 的循環群，$z^d = 1$ 有 gcd(d,uᵢ) 個解，

而 $z^{2^r d} = -1$（0≤r<tᵢ）有 2ʳgcd(d,uᵢ) 個解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

強偽質數底數的精確計數

非互質底數一定不是 liar。CRT 把可逆底數分解成各質數冪座標；強測試接受當且僅當所有座標的 d 次冪均為 1，或存在同一 r∈[0,s) 使所有座標的 $2^r d$ 次冪均為 −1。這些情況互斥，因此 liar 數 $L = A\left(1 + \sum_{r=0}^{t-1} 2^{kr}\right)$。其中 k 是相異質因數的個數。這是數量，不是機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

計數界：三個以上的質因數

當 k≥3，φ(n)=∏2^{t_i}u$_i$≥2^{kt}A。且 $\left(1+\sum_{r=0}^{t-1}2^{kr}\right)/2^{kt}\le 1/4$：t=1 時為 2/2^k≤1/4，t>1 時分子比對應幾何級數更小。故 L≤φ(n)/4≤(n−1)/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

計數界：恰有兩個質因數

若兩個 t_i 不同，$\phi(n) \geq 2^{2t+1}A$，且 $1 + \sum_{r=0}^{t-1} 4^r \leq 4^t/2$，因此 $L \leq \phi(n)/4$。若兩個 t_i 相同，而某個 $\gcd(d,u_i)<u_i$，或某個 $e_i>1$，因損失的奇因子至少為 3，$\phi(n) \geq 3 \cdot 2^{2t}A$，得到 $L \leq \phi(n)/4$。

剩下的情況是不同奇質數 p,q 各一次方、$t_1=t_2=t$ 且 u_1,u_2 均整除 d。此時 $s \geq t$：$p=1+2^t u_1$、$q=1+2^t u_2$，所以 $pq-1$ 可被 2^t 整除。故 $p-1$、$q-1$ 都整除 $pq-1$；模 $p-1$ 和模 $q-1$ 分別得 $p-1 \mid q-1$、$q-1 \mid p-1$，迫使 $p=q$，矛盾。故該情況不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

計數界：奇質數冪

若 $n=p^e$、$e\geq 2$，因 p 不整除 d，$A=\gcd(d,p^{e-1}(p-1)$ 的奇部$)$ 最多是 $p-1$ 的奇部。因 $t\leq v_2(p-1)$，$L=A\cdot 2^t\leq p-1\leq (p^e-1)/4$，最後一步對奇 $p\geq 3$、$e\geq 2$ 成立。

綜合三種情況，$L\leq (n-1)/4$。若從 $1,\cdots,n-1$ 均勻抽取，單輪誤判率 $\leq 1/4$；若從 $2,\cdots,n-2$ 均勻抽取，去掉必為 liar 的 1、$n-1$ 後誤判率仍 $\leq 1/4$。獨立重複 r 輪才得到 $\leq 4^{-r}$。質數不會被此測試誤判為合數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

固定合數誤判與事後機率

對每個固定奇合數，若每輪獨立且均勻取合法底數，k 輪均通過的機率不超過 $\varepsilon=4^{-k}$。要使 $\varepsilon\le 10^{-9}$，因 $4^{14}=268435456<10^{9}\le 4^{15}=1073741824$，依此通用上界保證目標時，最小可選 15 輪；這不是說所有輸入的實際誤判率都需要 15 輪才能達標。

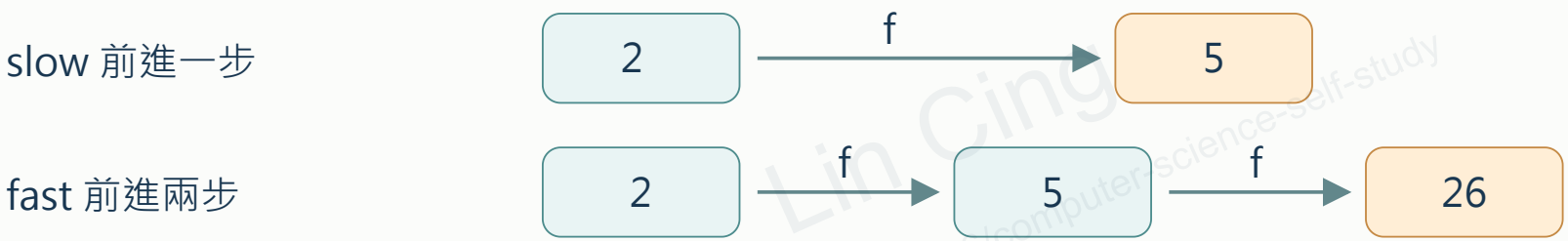
假設候選數分布的先驗質數率 $\pi=1/100$，測試對質數必通過，且對每個合數的通過率均 $\le\varepsilon$。由 Bayes 公式，$P(\text{合數}\mid\text{通過})\le(1-\pi)\varepsilon/[\pi+(1-\pi)\varepsilon]$。$k=15$ 時上界為 $99/(1073741824+99)\approx 9.22\times10^{-8}$。這是該先驗下的後驗上界，不是固定合數的假陽性機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pollard ρ 與 Floyd 碰撞

選 $f(x)=(x^2+1) \bmod n$，令 slow=f(slow)、fast=f(f(fast))，每輪 $d=\gcd(|slow-fast|,n)$。若 $1<d<n$，得到非平凡因數；若 d=n，本次未分離因數，重新選種子或函數；d=1 則繼續。找到 d 不表示 d 一定是質數，仍須遞迴分解或檢測。

Floyd 的雙速追蹤：$n=91$，$f(x)=x^2+1 \bmod 91$



$\gcd(|5-26|,91)=7$，找到因數 7

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pollard ρ 數例與啟發式

$n=91$，$x_0=2$。第一輪 $\text{slow}=f(2)=5$；$\text{fast}=f(f(2))=f(5)=26$；$\gcd(|5-26|,91)=\gcd(21,91)=7$，故 $91=7\cdot 13$。這是固定 f 與種子的一個成功例，不保證其他輸入或種子都成功。

如果序列模未知最小質因數 p 的行為像隨機映射，生日碰撞啟發給約 $O(\sqrt{p})$ 次迭代；這依賴模型，不是最壞情況保證，也不含每次模乘與 gcd 的 bit 成本。可設定迭代上限並重啟，不能把啟發式寫成必然終止的定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floyd 的三輪追蹤：8051

固定 $n=8051$、$x_0=2$、$f(x)=x^2+1 \bmod 8051$，slow 每輪走一步，fast 走兩步。每輪計算 $d=\gcd(|slow-fast|,8051)$。

輪	slow	fast	差的絕對值	d
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

第三輪 $677 \equiv 871 \equiv 95 \pmod{97}$；但它們模 83 分別是 13、41。因而 $\gcd(194,8051)=97$，且 $8051=97 \cdot 83$。最後兩因數可用試除至各自平方根確認為質數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Floyd 的三輪追蹤：8051 （續）

第三輪比較兩個模數分量

slow = 677		fast = 871
677 mod 97 = 95	=	871 mod 97 = 95
677 mod 83 = 13	≠	871 mod 83 = 41

同餘模 97，但不相同模 83；因此 gcd = 97

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checkpoint 是另一種排程

定義 $x_{i+1}=f(x_i)$。對 j=0,1,2,⋯，保存 $y=x_{2^j}$，讓 x 按序走過 $x_{2^j+1},\cdots,x_{2^{j+1}}$，每一步算 gcd(|x−y|,n)，比較後才更新下一個 checkpoint。若得到 1<d<n 則返回；若 d=n，重新選函數或種子；達工作上限仍無因數，只能回報本次未找到。

對有限函數圖，從 checkpoint 開始的序列終會進入長度 λ 的 cycle。若 y 已在 cycle 且隨後區間至少走 λ 步，就會遇到與 y 相同的分量。兩倍成長排程使用常數個整數狀態，與 Floyd 的 slow/fast 不是同一條追蹤表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checkpoint 的具體操作

同樣取 n=8051、x_0=2。依序得到 x_1=5、x_2=26、x_3=677、x_4=7474、x_5=2839、x_6=871、x_7=1848。

checkpoint y	依次比較的 x	gcd 結果
x_1=5	x_2=26	1
x_2=26	x_3=677、x_4=7474	1、1
x_4=7474	x_5=2839、x_6=871、x_7=1848	1、1、97

在 x_7 找到 97；因 97 是 8051 的非平凡因數，停止。本例的比較次序不同於 Floyd，雖然底層序列 x_i 相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

生日碰撞界從何而來？

模型：從大小為 p 的集合獨立、均勻抽取 k 次；若 k≤p，前 j 次互異時，第 j+1 次避開既有值的機率為 1−j/p。連乘得 $\Pr(\text{無碰撞}) = \prod_{j=0}^{k-1}(1 - j/p)$。利用 1−u≤$e^{-u}$，得到 $\Pr(\text{無碰撞}) \le e^{-k(k-1)/(2p)}$；k>p 時無碰撞機率為 0。

令 k 約為 c$\sqrt{\ }$p，指數約為 −c^2/2，故典型碰撞尺度為 $\sqrt{\ }$p。這是獨立均勻抽樣的精確模型；隨機映射的初次碰撞過程也有相同連乘式。固定多項式 f(x)=x^2+c 不能保證符合模型，因此 $\sqrt{\ }$p 是啟發式，並非無條件的最壞時間或每次重啟成功率保證。平衡半質數 p≈$\sqrt{\ }$n 時，對應 $n^{1/4}$ 次迭代的啟發尺度；每次模乘與 gcd 的 bit 成本須另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

獨立性與 Floyd 相遇的精確界

重複固定底數不會讓同一事件獨立；2047 對 base 2 每次均通過，再測多少次也如此。只有各輪依規定獨立抽樣，才可把每輪上界相乘。

有限函數圖從 x_0 出發，尾長 $\mu \geq 0$、環長 $\lambda \geq 1$。Floyd 第 k 輪比較 x_k 與 x_{2k}。取 k 為不小於 $\max(\mu,1)$ 的最小 λ 倍數，則兩者均在環上、索引差 k 是 λ 的倍數，因此相遇，且 $k \leq \mu+\lambda$。這是函數圖上必然的相遇界；遇到相同值不必然得到非平凡因數。要估為 $O(\sqrt{p})$ 還需近似隨機映射的額外假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

gcd=n 的失敗案例

$n=15$，$x_0=0$，$f(x)=x^2+2 \bmod 15$。第一輪 slow=2、fast=6，$\gcd(|2-6|,15)=1$。第二輪 slow=6、fast=f(f(6))=f(8)=6，$\gcd(0,15)=15$。兩指標在模 n 的整個狀態相遇，回傳 n 不是成功的非平凡因數。

此時須換種子或函數重新嘗試，或在工作預算用盡時回報本次未找到因數；不能將失敗推論為 n 是質數，也不能回報 15 作為 15 的非平凡分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA 建鑰

取不同質數 p=11、q=13，n=143，$\phi(n)=120$。選 e=7 且 gcd(7,120)=1，求 d=103，因 $7\cdot 103=721\equiv 1 \pmod{120}$。公開 (n,e)，保密 d、p、q。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA 加密與解密

訊息代表元 $0 \le M < n$。以 $M=9$ 為例，$C=9^7 \bmod 143=48$；解密 $48^{103} \bmod 143=9$。這是教學用小數值，無安全性。

- $p=11, q=13, n=143$
- $e=7, d=103$
- $9 \rightarrow 48 \rightarrow 9$
- encrypt: $9^7 \bmod 143 = 48$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA 正確性

ed=1+kφ(n)。對 p，若 p|M 則兩邊皆為 0；否則由費馬定理 $M^{ed} \equiv M \pmod{p}$。對 q 同理；互質 CRT 得 $M^{ed} \equiv M \pmod{n}$，包含非互質訊息。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA 對非 unit 也能還原

取不同質數 p=11、q=17；n=187，φ(n)=160，e=7，d=23，因為 7・23=161≡1 (mod 160)。選訊息 m=11，雖 gcd(11,187)=11，仍是合法的模 n 訊息。

加密：$11^2 \equiv 121$、$11^4 \equiv 55$，故 $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$。解密的平方鏈為 $88^2 \equiv 77$、$88^4 \equiv 132$、$88^8 \equiv 33$、$88^{16} \equiv 154$；23=16+4+2+1，154・132≡132，132・77≡66，66・88≡11。故 $88^{23} \equiv 11 \pmod{187}$。

非 unit 訊息：逐質數分量驗證

模 11：訊息是 0	m=0	—7次方→	c=0	—23次方→	m′=0
模 17：訊息可逆	m=11	—7次方→	c=3	—23次方→	m′=11

兩個分量均還原原值，CRT 唯一還原 11 mod 187

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為何非 unit 的 RSA 等式成立？

設 $ed \equiv 1 \pmod{(p-1)(q-1)}$。對每個質數因子 $r \in \{p,q\}$：若 $r \mid m$，則 $m^{ed} \equiv m \equiv 0 \pmod{r}$；否則由 Fermat 定理及 $r-1 \mid ed-1$，得 $m^{ed} \equiv m \pmod{r}$。兩個分量均相同，由 CRT 得 $m^{ed} \equiv m \pmod{pq}$。

本例模 11 分量是 $11 \to 0$、$88 \to 0$；模 17 分量是 $11 \to 11$、$88 \to 3$，而 $3^{23} \equiv 11$。兩分量組合唯一還原 $11 \pmod{187}$。上述論證假設 p、q 是相異質數、e,d 為正整數且乘積符合條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

裸 RSA 的可塑性

不加編碼的 $E(m)=m^e \bmod n$ 是確定性映射，而且 $E(m_1)E(m_2)\equiv E(m_1m_2) \pmod n$。因此若 $c=E(m)$，任何 $r\in U(n)$ 都可計算 $c'=c\cdot r^e \bmod n$；它會解密成 $mr \bmod n$。這個代數可塑性不依賴知道 m 或私鑰。

例 $n=187$、$e=7$、$c=88$、$r=2$：$2^7\equiv 128$，$c'\equiv 88\cdot 128\equiv 44$；$44^{23}\equiv 22$，正是 $2\cdot 11$。這不表示攻擊者一定在任何實際協定中得到明文，但揭示了直接使用裸冪無法提供所需安全性；實務加密必須採用經標準化的隨機化編碼與適當安全模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

裸 RSA 簽章的另一種缺陷

若簽章定義成 $\sigma = m^d \bmod n$，驗證只檢查 $\sigma^e \equiv m \pmod n$，攻擊者可先任選 σ，再設 $m = \sigma^e \bmod n$，得到可通過驗證的訊息／簽章對，無需私鑰。此構造讓攻擊者選擇 σ，通常不能讓他同時指定任意目標訊息；它證明的是在此訊息空間與裸驗證規則下可產生一組未經簽署的有效對。

例 $n=55$、$e=3$，選 $\sigma=12$，則 $m = 12^3 \bmod 55 = 23$，因此驗證 $12^3 \equiv 23$ 成功。這是簽章驗證缺陷，與前頁討論的加密密文可塑性是不同的安全性問題；實際簽章還需經過合適的雜湊、編碼與嚴格驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

範圍與實務

本章為數學演算法，實際 RSA 需標準化編碼與隨機填充、安全參數及防側通道實作。質數測試的機率保證依隨機抽樣條件成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 31. MIT 6.046J Fall 2004 Lecture 7, p. 33, and Fall 2005 Lecture 8, p. 13: inverses modulo a prime.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study